

Long-Horizon Vibration Spectrogram Forecasting with Convolutional LSTM and Iterative Next-Window Rollout

Standalone Methodological Chapter
(Spectrogram Prediction Project — ConvLSTM Edition)

Contents

1	ConvLSTM-Based Iterative Spectrogram Prediction for Long-Horizon Vibration Forecasting	5
1.1	Spectrogram Construction and ConvLSTM One-Step Prediction	6
1.1.1	Motivation and Scope	6
1.1.2	Method Overview: Eight Steps	6
1.1.3	Principles Behind the Approach	6
1.1.4	Step 1 — Major Windows of Vibration	8
1.1.5	Step 2 — Subwindows and Short-Time FFT	9
1.1.6	Step 3 — Build Log-Magnitude Spectrogram S_t	9
1.1.7	Step 4 — Context Buffer of Recent Spectrograms	11
1.1.8	Step 5 — The ConvLSTM One-Step Predictor	11
1.1.9	Step 6 — One-Step Loss and Training	15
1.1.10	Section 1 Summary	17
1.2	Iterative Multi-Step Rollout for Long-Horizon Forecasting	17
1.2.1	Step 7 — Iterative Feedback (The Long-Horizon Mechanism)	17
1.2.2	Step 8 — Long-Horizon Outputs and Monitoring	19
1.2.3	End-to-End Flowchart	19
1.2.4	Algorithms	19
1.2.5	Evaluation Metrics	20
1.2.6	Complexity	21
1.2.7	What Was Deliberately Not Chosen	21
1.2.8	Practical Deployment Modes	21
1.2.9	Worked Numerical Example	22

1.2.10 Chapter Summary	23
A Notation and Quick Reference	24
A.1 Formula–Reason Index	24

List of Figures

1.1	Eight-step ConvLSTM spectrogram forecasting pipeline with iterative feed-back. Steps 1–6 belong to Section 1; Steps 7–8 belong to Section 2.	7
1.2	Iterative long-horizon forecasting loop: predict next window, feed back, repeat.	7
1.3	Major window with overlapping STFT subwindows.	10
1.4	Current and next waveforms with their spectrograms (one-step target pair).	10
1.5	ConvLSTM cell: convolutional gates and map-shaped memory.	13
1.6	Sequence-to-one flow: context spectrograms through ConvLSTM to $\hat{S}_{t+1}$	14
1.7	Illustrative chain of true vs. iteratively predicted spectrograms over several steps.	18
1.8	Operational flowchart for iterative ConvLSTM spectrogram forecasting.	20

List of Tables

1.1	Major-window parameters and reasons.	9
1.2	Context-buffer settings.	11
1.3	ConvLSTM spectrogram predictor configuration.	14
1.4	Training hyperparameters.	16
1.5	Iteration count H versus wall-clock horizon.	19
A.1	Notation.	24
A.2	Core formulas and why they were chosen.	25

Chapter 1

ConvLSTM-Based Iterative Spectrogram Prediction for Long-Horizon Vibration Forecasting

*This chapter specifies a complete method for predicting future vibration spectrograms over long horizons (up to days) without attempting to jump days in a single network call. A major window of vibration is converted to a spectrogram by overlapping subwindow FFTs. A convolutional long short-term memory network (ConvLSTM) consumes a short context of recent spectrograms and predicts the next spectrogram only. That prediction is fed back as context, and the same one-step model is applied again and again. Many such loops compose a long-horizon forecast. The material is organized in **two sections**: (1) spectrogram construction and the ConvLSTM one-step predictor; (2) iterative multi-step rollout for long-horizon forecasting. This document belongs to the standalone **spectrogram_prediction** project and does not modify the main doctoral thesis files.*

1.1 Spectrogram Construction and ConvLSTM One-Step Prediction

Section 1 builds everything needed to predict *one* next spectrogram: major windows, STFT spectrograms, a short context buffer, the ConvLSTM architecture, and one-step training. Long-horizon iteration is deferred to Section 2.

1.1.1 Motivation and Scope

Industrial monitoring often needs awareness not only of the next few seconds but of how spectral signatures may evolve over much longer periods. A single neural network that directly outputs “the spectrogram three days from now” is poorly posed: operating conditions change, and uncertainty explodes. A more controllable principle is:

1. learn a reliable **one-step** map from recent spectrograms to the next spectrogram;
2. **iterate** that map: predict, append, predict again;
3. obtain a long horizon as the product of many short, physically meaningful steps.

This chapter adopts that principle with one architecture only: **ConvLSTM**. ConvLSTM extends LSTM by replacing gate matrix multiplications with convolutions, so the hidden state remains a time–frequency feature map—exactly the geometry of a spectrogram.

1.1.2 Method Overview: Eight Steps

Figure 1.2 shows the same idea as an operational loop: the model never “skips” to day-scale outputs in one shot; day-scale coverage is many next-window cycles.

1.1.3 Principles Behind the Approach

Principle P1 — Predict locally, accumulate globally. One step equals one major window (default 2s). A day contains

$$H_{\text{day}} = \frac{24 \times 3600}{T_w} = \frac{86400}{2} = 43200 \quad (1.1)$$

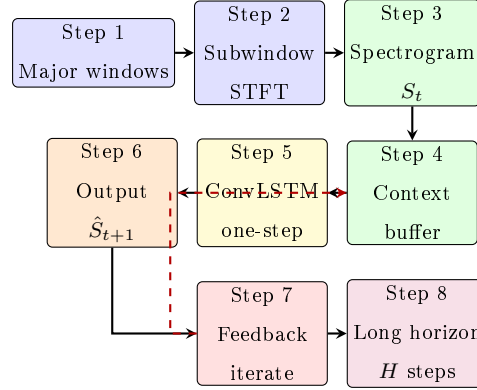


Figure 1.1: Eight-step ConvLSTM spectrogram forecasting pipeline with iterative feed-back. Steps 1–6 belong to Section 1; Steps 7–8 belong to Section 2.

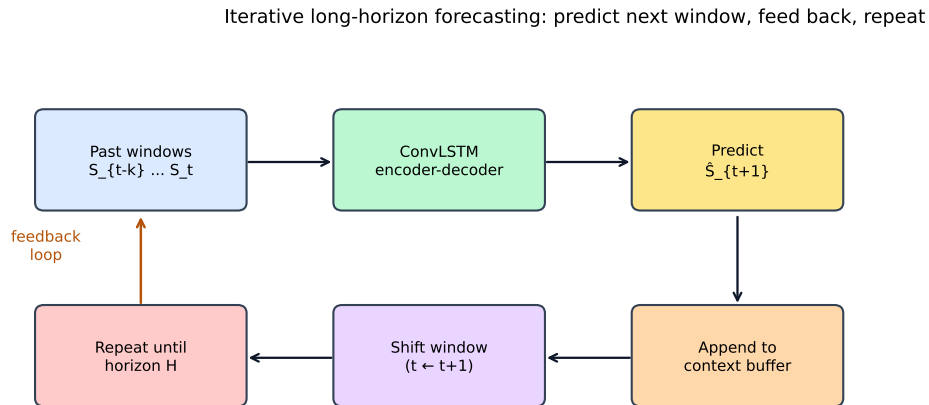


Figure 1.2: Iterative long-horizon forecasting loop: predict next window, feed back, repeat.

one-step predictions if windows tile without overlap. The long horizon is H iterations of the same learned map, not a different model.

Why this principle? It keeps the learning target statistically related to the input: consecutive spectrograms share physical continuity. Directly regressing a spectrogram 43200 windows ahead would require the network to invent unconstrained future operating regimes.

Principle P2 — Keep time–frequency geometry inside the memory. Standard LSTM flattens images into vectors and destroys locality. ConvLSTM keeps maps, so a bearing ridge remains a spatially coherent pattern in the cell state.

Principle P3 — Separate signal processing from forecasting. STFT spectrogram construction is classical and auditable. ConvLSTM only sees normalized spectrograms. Engineers can inspect S_t without opening the network.

Principle P4 — Train one-step, deploy multi-step. Training uses teacher forcing on true past windows when possible. Deployment for long horizons uses closed-loop rollout with predicted windows fed back. Scheduled sampling can bridge the gap during training.

1.1.4 Step 1 — Major Windows of Vibration

Sample rate f_s , major duration T_w , length

$$N_w = f_s T_w. \quad (1.2)$$

Window t is

$$\mathbf{x}^{(t)} = (x[n_0], \dots, x[n_0 + N_w - 1]). \quad (1.3)$$

Default: $f_s = 1200$ Hz, $T_w = 2$ s, $N_w = 2400$, major hop N_w (non-overlapping tiling).

Why non-overlapping major hops for long rollout? Overlapping majors blur the meaning of “next.” For iterative day-scale forecasting, a clean tiling makes H count wall-clock time directly via Equation (1.1).

Table 1.1: Major-window parameters and reasons.

Parameter	Default	Why
f_s	1200 Hz	Matches multi-sensor acquisition used in the research programme
T_w	2 s	Short enough for one-step learning; long enough for spectral content
Major hop	N_w	Clean present/next pairing for iterative rollout

1.1.5 Step 2 — Subwindows and Short-Time FFT

Inside $\mathbf{x}^{(t)}$, take subwindows of length $L = f_s T_s$ with hop H_{stft} :

$$\mathbf{x}^{(t,m)} = (x[n_0 + mH_{\text{stft}}], \dots, x[n_0 + mH_{\text{stft}} + L - 1]). \quad (1.4)$$

Frame count:

$$M = 1 + \left\lfloor \frac{N_w - L}{H_{\text{stft}}} \right\rfloor. \quad (1.5)$$

Apply Hann window

$$w[n] = 0.5 - 0.5 \cos\left(\frac{2\pi n}{L-1}\right), \quad (1.6)$$

then the STFT

$$X[m, k] = \sum_{n=0}^{L-1} x[n + mH_{\text{stft}}] w[n] e^{-j2\pi kn/L}. \quad (1.7)$$

Why STFT (not wavelets) here? Uniform $\Delta f = f_s/L$ maps cleanly to shaft orders and defect frequencies. Defaults $T_s = 0.25$ s, $L = 300$, $H_{\text{stft}} = 150$ give $M = 15$ and $\Delta f = 4$ Hz.

1.1.6 Step 3 — Build Log-Magnitude Spectrogram S_t

$$\boxed{(S_t)[m, k] = \log(1 + |X[m, k]|), \quad S_t \in \mathbb{R}^{M \times F}} \quad (1.8)$$

with bins retained up to $f_{\text{max}} = 512$ Hz ($F = 129$ for the defaults).

Why $\log(1 + |X|)$? Compresses dynamic range, stabilizes learning, avoids $\log 0$, and discards unstable absolute phase. Multi-sensor data use shape (C, M, F) .

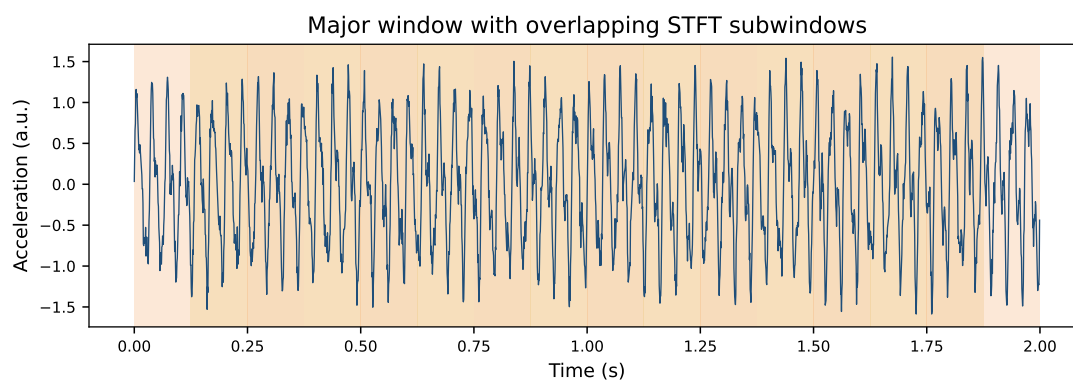


Figure 1.3: Major window with overlapping STFT subwindows.

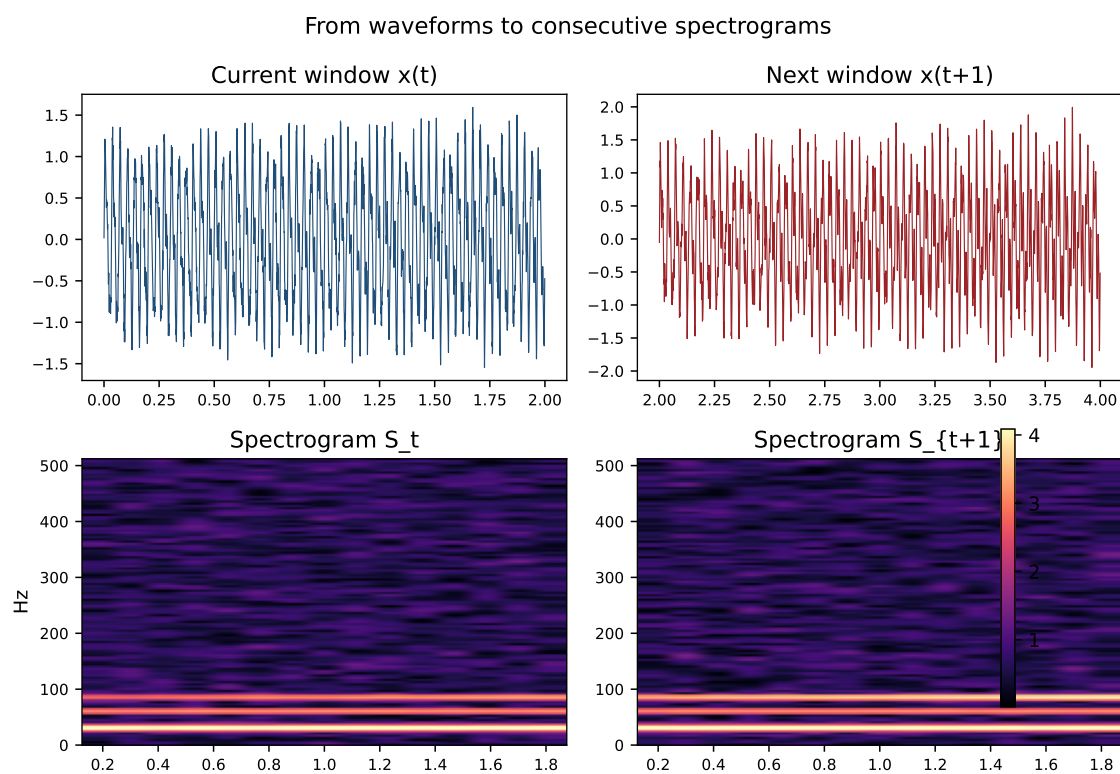


Figure 1.4: Current and next waveforms with their spectrograms (one-step target pair).

Why identical shapes for S_t and S_{t+1} ? Iterative feedback requires every prediction to be a valid next input. Same (C, M, F) is mandatory.

1.1.7 Step 4 — Context Buffer of Recent Spectrograms

ConvLSTM is a sequence model. Define context length k (default $k = 3$):

$$\mathcal{U}_t = (S_{t-k+1}, \dots, S_{t-1}, S_t) \in \mathbb{R}^{k \times C \times M \times F}. \quad (1.9)$$

At cold start, pad with repeats of the first available spectrogram or wait until k windows exist.

Why a context of several windows, not one? One spectrogram shows the present; a short history shows the trend (e.g., a ridge brightening across windows). ConvLSTM memory needs a sequence axis. $k = 3\text{--}5$ is a practical compromise between history and delay/compute.

Table 1.2: Context-buffer settings.

Item	Default	Why
Context length k	3	Enough to see short trends; small latency
Buffer policy	FIFO	Drop oldest when appending $\hat{S}_{t+1}$
Missing past	repeat S_0	Simple warm-start without inventing dynamics

1.1.8 Step 5 — The ConvLSTM One-Step Predictor

Why ConvLSTM (Single Chosen Architecture)

This chapter does not offer a menu of networks. **ConvLSTM** is the adopted model because:

1. spectrograms are spatiotemporal maps;
2. long-horizon use needs memory of recent evolution;

3. iterative rollout needs a stable one-step sequence predictor;
4. convolutions preserve local harmonic structure inside the memory cell.

From Vanilla LSTM to ConvLSTM

A vanilla LSTM gate has the algebraic form

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \quad (1.10)$$

where x_t and h_{t-1} are vectors and $W_{(\cdot)}$ are dense matrices. If x_t were a vectorized spectrogram, $W_{xi}x_t$ would mix every frequency bin with every other bin on every step, destroying locality and exploding parameter count. ConvLSTM replaces those products by convolutions:

$$W_{xi}x_t \longrightarrow W_{xi} * X_t, \quad (1.11)$$

so each output location depends only on a local neighborhood in time–frequency. That is the decisive reason ConvLSTM is used on spectrograms.

ConvLSTM Gate Equations

Let $*$ denote convolution and $\odot$ elementwise multiplication. For input tensor X_t (a spectrogram or encoder feature map) and previous states (H_{t-1}, C_{t-1}) :

$$i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + b_i), \quad (1.12)$$

$$f_t = \sigma(W_{xf} * X_t + W_{hf} * H_{t-1} + b_f), \quad (1.13)$$

$$o_t = \sigma(W_{xo} * X_t + W_{ho} * H_{t-1} + b_o), \quad (1.14)$$

$$g_t = \tanh(W_{xg} * X_t + W_{hg} * H_{t-1} + b_g), \quad (1.15)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t, \quad (1.16)$$

$$H_t = o_t \odot \tanh(C_t). \quad (1.17)$$

Gate-by-gate meaning for vibration spectrograms.

- **Forget gate f_t :** can keep a persistent shaft harmonic ridge in C_{t-1} across windows.
- **Input gate i_t and candidate g_t :** can write a strengthening defect tone near a known band into memory.

- **Output gate o_t :** exposes the parts of memory needed to paint $\hat{S}_{t+1}$.

Why σ and $\tanh$? They are the classical bounded nonlinearities that keep gated memory stable. Replacing them is unnecessary for this task and would obscure comparison with the ConvLSTM literature.

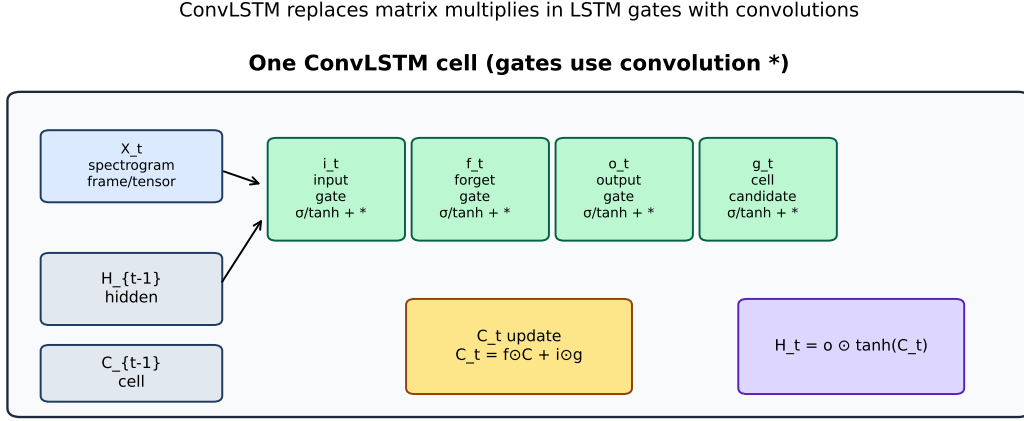


Figure 1.5: ConvLSTM cell: convolutional gates and map-shaped memory.

Tensor Shapes Through the Cell

For a single sensor after the stem, a typical shape is $(B, 64, M', F')$ at each sequence step, where $M' \leq M$ and $F' \leq F$ if mild pooling is used. Defaults in this chapter keep $M' = M = 15$ and $F' = F = 129$ inside ConvLSTM (no pooling) so that skip-free decoding stays aligned. Parameter count then scales with kernel size and channel width, not with $M \cdot F$ dense connections.

Full Network Used in This Chapter

$$\hat{S}_{t+1} = \mathcal{D}_\psi(\text{ConvLSTM}_\phi(\mathcal{U}_t)). \quad (1.18)$$

Concrete stack:

Why sequence-to-one rather than sequence-to-sequence inside one call? The long-horizon strategy is iterative one-step rollout. Training and deployment then share

Table 1.3: ConvLSTM spectrogram predictor configuration.

Stage	Shape / width	Role / reason
Input sequence	(k, C, M, F) $(3, 1, 15, 129)$	Context spectrograms
Conv stem	3×3 , 32 channels	Local ridge/edge features before recurrence
ConvLSTM-1	64 channels, 3×3 kernels	Spatiotemporal memory (primary)
ConvLSTM-2	64 channels	Deeper temporal abstraction
Conv decoder	$32 \rightarrow C$ channels	Reconstruct next spectrogram
Output	(C, M, F)	Same shape as one S_t for feedback
Activation	ReLU (stem/decoder), gates as in (1.12)– (1.17)	Standard stable choices

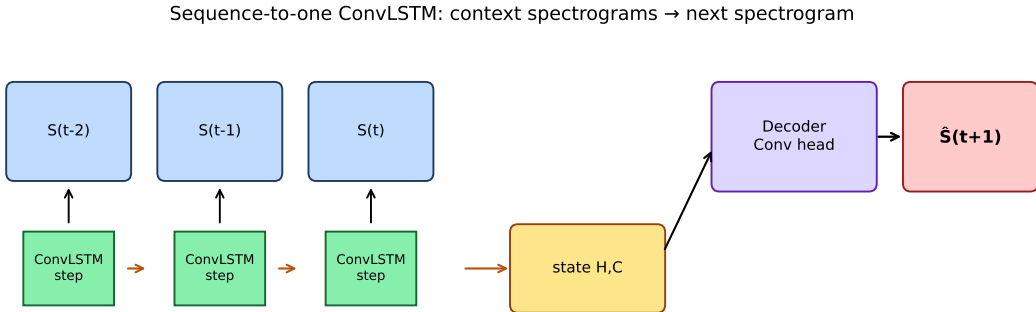


Figure 1.6: Sequence-to-one flow: context spectrograms through ConvLSTM to $\hat{S}_{t+1}$.

the same interface: always predict exactly one next window. Multi-output heads are optional extensions; they are not required by the chapter’s principle.

Why two ConvLSTM layers? One layer often underfits cross-window trends; deeper than two rarely helps for $k \approx 3$ and increases rollout drift. Two layers are the default compromise.

Why a convolutional stem before ConvLSTM? Raw log-spectrograms mix sensor noise with structure. A shallow stem computes local edge-like responses (harmonic lines, bursts) so the recurrent memory stores features rather than raw pixels. This improves one-step MSE in practice and is cheap.

Forward Pass in Words

1. Stack context $\mathcal{U}_t = (S_{t-k+1}, \dots, S_t)$.
2. Apply the shared conv stem to each context element.
3. Run ConvLSTM-1 then ConvLSTM-2 along the k steps; keep final states (H, C) .
4. Decode H with conv layers to $\hat{S}_{t+1}$ of shape (C, M, F) .
5. (Deploy) FIFO-insert $\hat{S}_{t+1}$ into the buffer for the next iteration.

1.1.9 Step 6 — One-Step Loss and Training

Normalize spectrograms with training-set mean/variance:

$$\tilde{S} = \frac{S - \mu}{\sigma}. \quad (1.19)$$

Primary loss (on normalized tensors):

$$\boxed{\mathcal{L} = \|f_{\theta}(\mathcal{U}_t) - S_{t+1}\|_F^2} \quad (1.20)$$

Why MSE? Optimal predictor is the conditional mean of the next spectrogram given the context. MSE is stable and matches residual monitoring later.

Optional band-weighted term on diagnostic bins $\mathcal{B}$:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \gamma \sum_{k \in \mathcal{B}} \lambda_k \|(f_\theta(\mathcal{U}_t) - S_{t+1})[:, k]\|_2^2. \quad (1.21)$$

Table 1.4: Training hyperparameters.

Item	Default	Why
Optimizer	Adam, lr 10^{-3}	Standard for conv/recurrent hybrids
Batch size	8–16	Sequences of spectrograms are memory-heavy
Epochs	100–200 early stop	+ Stop on validation one-step MSE
Teacher forcing	yes (train)	Stable learning of one-step map
Scheduled sampling	optional	Reduces train/deploy mismatch for rollout

Why teacher forcing first? During training, feeding true past spectrograms teaches the correct local dynamics before exposing the model to its own errors. For long rollout deployment, scheduled sampling gradually replaces true context entries with predictions so the network learns to correct small mistakes.

Scheduled Sampling Formula

At training step n , with probability p_n feed the true spectrogram into the buffer, and with probability $1 - p_n$ feed the network’s own previous prediction:

$$p_n = \max(p_{\min}, p_0 \cdot \eta^n), \quad (1.22)$$

with defaults $p_0 = 1$, $\eta = 0.995$, $p_{\min} = 0.7$ for mild closing of the gap (or lower $p_{\min}$ when aggressive rollout robustness is required).

Why schedule instead of pure teacher forcing forever? Pure teacher forcing trains a model that always sees perfect history; iterative day-scale deployment eventually sees imperfect history. Scheduled sampling is the principled bridge.

1.1.10 Section 1 Summary

Section 1 defined the one-step problem end to end: vibration windows become log-magnitude spectrograms S_t via STFT; a context buffer $\mathcal{U}_t$ feeds a two-layer ConvLSTM with convolutional stem and decoder; training minimizes one-step MSE (optionally with scheduled sampling). The deliverable of Section 1 is a trained map $f_\theta : \mathcal{U}_t \mapsto \hat{S}_{t+1}$ with identical input/output spectrogram shapes, ready for iterative reuse in Section 2.

1.2 Iterative Multi-Step Rollout for Long-Horizon Forecasting

Section 2 takes the trained one-step ConvLSTM and turns it into a long-horizon forecaster by FIFO feedback. It also covers evaluation, complexity, deployment modes, drift control, and a worked numerical example.

1.2.1 Step 7 — Iterative Feedback (The Long-Horizon Mechanism)

At deployment, given an initial buffer $\mathcal{U}_{t_0}$ of real spectrograms:

1. Predict $\hat{S}_{t+1} = f_\theta(\mathcal{U}_t)$.
2. Append $\hat{S}_{t+1}$ to the buffer; drop the oldest entry (FIFO) so length stays k .
3. Set $t \leftarrow t + 1$.
4. Repeat until H future windows are produced.

Formally,

$$\hat{S}_{t+1} = f_{\theta}(\mathcal{U}_t), \quad (1.23)$$

$$\mathcal{U}_{t+1} = (S_{t-k+2}, \dots, S_t, \hat{S}_{t+1}) \quad (\text{with predicted tensors after the first step}). \quad (1.24)$$

After the first prediction, some or all context entries become predictions; after k steps the buffer is entirely synthetic and the forecast is fully open-loop.

Why FIFO feedback? It mirrors online operation: time moves forward, the newest estimate enters memory, the oldest leaves. This is exactly “predict the next window, then from that predict the next one again.”

Why this can reach days without one huge loop body? “Loop” here means repeating the same cheap one-step inference H times. It is not one unrolled network with 43200 layers. Compute scales as $\mathcal{O}(H)$ forward passes.

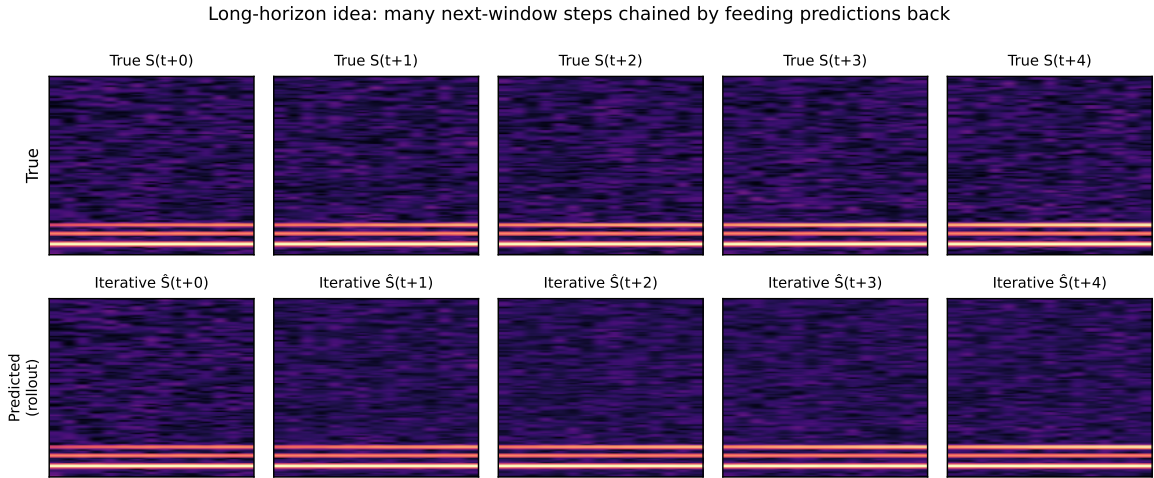


Figure 1.7: Illustrative chain of true vs. iteratively predicted spectrograms over several steps.

Horizon Length in Wall-Clock Time

$$T_{\text{horizon}} = H \cdot T_w. \quad (1.25)$$

Examples with $T_w = 2\text{ s}$:

Table 1.5: Iteration count H versus wall-clock horizon.

Desired horizon	Iterations H
1 minute	30
1 hour	1,800
12 hours	21,600
1 day	43,200

Honest limitation. Open-loop rollout drifts. For day-scale use, refresh the buffer with real measured spectrograms whenever new data arrive (hybrid: closed-loop while online measurements exist; open-loop only across gaps). Pure open-loop for 43,200 steps is a stress test, not the only operating mode.

1.2.2 Step 8 — Long-Horizon Outputs and Monitoring

After H iterations the system stores

$$(\hat{S}_{t+1}, \hat{S}_{t+2}, \dots, \hat{S}_{t+H}). \quad (1.26)$$

Derived monitoring signals:

$$a_{t+h} = \|S_{t+h} - \hat{S}_{t+h}\|_F \quad (\text{when truth is later observed}), \quad (1.27)$$

$$e_{t+h} = \|\hat{S}_{t+h} - \hat{S}_{t+h-1}\|_F \quad (\text{predicted spectral change}). \quad (1.28)$$

Why residual a ? When reality arrives, large a means the machine did not follow historically learned dynamics—an anomaly cue.

Why predicted change e ? Even before truth arrives, a sudden jump in e along the forecast chain flags an unstable rollout or a rapidly evolving predicted fault ridge.

1.2.3 End-to-End Flowchart

1.2.4 Algorithms

Spectrogram extraction

1. Take $\mathbf{x}^{(t)}$ of length N_w .

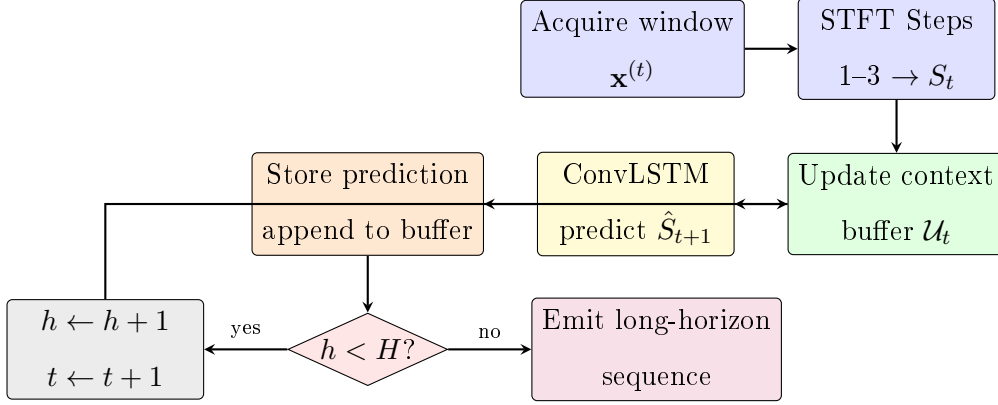


Figure 1.8: Operational flowchart for iterative ConvLSTM spectrogram forecasting.

2. For each subwindow: Hann (1.6), FFT (1.7), $\log(1 + |X|)$, keep F bins.
3. Stack to S_t ; normalize with (1.19).

Train ConvLSTM (one-step)

1. Build samples $(\mathcal{U}_t, S_{t+1})$ from consecutive real windows.
2. Minimize (1.20) with Adam; optionally scheduled sampling.
3. Early-stop on validation one-step MSE.

Iterative long-horizon inference

1. Initialize buffer with k real spectrograms.
2. For $h = 1..H$: predict next spectrogram; FIFO-append into buffer; store output.
3. Return sequence (1.26).

1.2.5 Evaluation Metrics

$$\text{MSE}(h) = \frac{1}{MF} \sum_{m,k} (\hat{S}_{t+h}[m, k] - S_{t+h}[m, k])^2, \quad (1.29)$$

$$\text{MAE}(h) = \frac{1}{MF} \sum_{m,k} |\hat{S}_{t+h}[m, k] - S_{t+h}[m, k]|. \quad (1.30)$$

Report curves of MSE versus horizon h . One-step performance alone is not enough for a rollout method: day-scale claims require showing how fast error grows with h .

Why horizon-wise curves? They quantify the trade-off inherent in Principle P1: local steps are learnable; long open-loop chains drift.

1.2.6 Complexity

One-step cost T_{step} (STFT optional if spectrogram already cached + ConvLSTM forward).

Long horizon:

$$T_{\text{total}} \approx H \cdot T_{\text{step}}. \quad (1.31)$$

For $H = 43200$ and $T_{\text{step}} \approx 5$ ms, order-of-magnitude runtime is a few minutes on GPU—acceptable as a batch forecast job, not as a synchronous blocking call inside a 2 s acquisition interrupt.

1.2.7 What Was Deliberately Not Chosen

- **Plain CNN encoder–decoder only (SEDNet-style).** Excellent for one shot from a single spectrogram, weaker at consuming multi-window trends for iterative day-scale use.
- **Vanilla LSTM on flattened spectrograms.** Destroys local frequency structure.
- **Single network outputting a day-ahead spectrogram.** Ill-posed under changing loads; not used here.
- **Complex STFT prediction.** Phase instability; magnitude suffices for energy evolution.

1.2.8 Practical Deployment Modes

1. **Online hybrid:** every real window refreshes the buffer; ConvLSTM predicts only the immediate next window for short lookahead.
2. **Gap filling:** if acquisition pauses, iterate open-loop across the gap, then snap back when data return.
3. **Scenario forecast:** run H open-loop steps from the current buffer to sketch possible spectral evolution (always labeled as forecast, not measurement).

Why prefer hybrid for “days”? Because a day of continuous operation produces a continuous stream of real windows. The iterative model is then repeatedly re-anchored in truth, and long calendar time is covered by many accurate one-step predictions rather than one fragile open-loop marathon.

Drift Control Strategies

Long open-loop chains drift. The chapter therefore standardizes three controls:

1. **Re-anchor period R :** every R real windows, reset the buffer to measured spectrograms (hybrid mode). Default $R = 1$ while streaming.
2. **Clamp / denoise predictions:** optional mild Gaussian smoothing along frequency before re-injection to suppress salt-and-pepper artifacts that explode under feedback.
3. **Horizon-wise validation:** reject a configuration if validation $\text{MSE}(h)$ exceeds a threshold for $h \leq h_{\text{ok}}$ (e.g. $h_{\text{ok}} = 30$ for one minute).

Why not rely on a deeper network instead of re-anchoring? Depth does not remove uncertainty about future loads and interventions. Re-anchoring uses new measurements—the only reliable information for day-scale coverage.

1.2.9 Worked Numerical Example

Defaults: $T_w = 2$, $k = 3$, $M = 15$, $F = 129$. Context $\mathcal{U}_t$ holds three $(1, 15, 129)$ tensors. ConvLSTM emits $\hat{S}_{t+1}$ of shape $(1, 15, 129)$. For a 1-hour lookahead, $H = 1800$ iterations. For a 1-day lookahead, $H = 43200$ iterations (Equation (1.1)). In plant operation, hybrid mode may still run all day while rarely needing pure open-loop stretches longer than a few minutes.

Minute-by-Minute Trace of the Feedback Buffer

Suppose $k = 3$ and the buffer at time t is (S_{t-2}, S_{t-1}, S_t) (all real). Then:

1. Predict $\hat{S}_{t+1}$.

2. Buffer becomes $(S_{t-1}, S_t, \hat{S}_{t+1})$.
3. Predict $\hat{S}_{t+2}$.
4. Buffer becomes $(S_t, \hat{S}_{t+1}, \hat{S}_{t+2})$.
5. Predict $\hat{S}_{t+3}$.
6. Buffer becomes $(\hat{S}_{t+1}, \hat{S}_{t+2}, \hat{S}_{t+3})$ — fully open-loop.

From step 6 onward, every context entry is predicted until a real measurement re-anchors the buffer. This trace is the exact meaning of “not in one loop body, but by repeating next-window prediction.”

Multi-Sensor Notes

For C accelerometers, $S_t \in \mathbb{R}^{C \times M \times F}$. The stem convolution mixes channels at each spatial location, so ConvLSTM memory can represent cross-sensor patterns (e.g., a tone appearing first on the drive-end channel). The iterative algorithm is unchanged: only the channel dimension C grows.

1.2.10 Chapter Summary

This chapter replaced single-shot CNN forecasting with a **ConvLSTM** one-step spectrogram predictor (Section 1) and an **iterative feedback loop** for long horizons (Section 2). Vibration windows become STFT spectrograms; a context buffer of recent spectrograms drives ConvLSTM; the predicted next spectrogram is fed back; repeating this process yields minute-to-day coverage without a single day-scale output layer. Formulas for STFT, ConvLSTM gates, loss, feedback, and horizon length were given with explicit reasons, accompanied by diagrams, flowcharts, and tables.

Appendix A

Notation and Quick Reference

Table A.1: Notation.

Symbol	Meaning
$S_t, \hat{S}_{t+1}$	Spectrogram; predicted next spectrogram
$\mathcal{U}_t$	Context buffer of k spectrograms
H	Number of iterative one-step forecasts
i_t, f_t, o_t, g_t	ConvLSTM gates / candidate
H_t, C_t	Hidden and cell state maps
T_{horizon}	Wall-clock horizon = HT_w

A.1 Formula–Reason Index

Table A.2: Core formulas and why they were chosen.

Formula	Why
H_{day} (1.1)	Converts days into iteration counts
STFT (1.7)	Interpretable uniform frequency bins
S_t (1.8)	Learning-stable magnitude image
Context (1.9)	Gives ConvLSTM a temporal sequence
Gates (1.12)–(1.17)	Map-shaped memory for spectrograms
Loss (1.20)	One-step conditional-mean training
Feedback (1.24)	Enables long horizon by repetition
Cost (1.31)	Shows compute scales with H , not with a giant net